

GitHub Talent Hunt Agent

From prototype to an LLM-powered, production-grade sourcing platform

Architecture, design rationale, and production hardening

Field	Value
Document	Engineering and Production-Readiness Report
Stack	Python 3.12, Flask, Gunicorn, OpenAI / AWS Bedrock, Docker
Target platform	AWS (ECS Fargate behind an Application Load Balancer)
Date	30 June 2026
Author	Nik

This report is written to be read end-to-end in an interview setting: each section explains not only *what* was built but *why*, with the trade-offs that were considered.

1. Executive Summary

The original project was a clever prototype: a Flask app that searched GitHub, scored users with a keyword heuristic, and rendered a polished results page. A production review surfaced blockers around security (debug mode always on), abuse resistance (unbounded search work on a public endpoint), ranking quality (substring skill matching), and operability (raw error leakage, unpinned dependencies).

This iteration resolves those findings in severity order and upgrades the core from a static heuristic into an **LLM-powered agent**. The agent plans a focused GitHub query from a natural-language job description, enriches candidate profiles in parallel, and evaluates the most promising people with a language model, while keeping the deterministic heuristic as an always-available fallback.

- **Smarter:** LLM query planning and candidate evaluation with transparent reasoning.
- **Safer:** rate limiting, input clamping, security headers, sanitized errors, secrets hygiene.
- **Operable:** Unicorn, Docker, health checks, structured logging, AWS-ready configuration.
- **Resilient:** retries and rate-limit handling for GitHub; graceful degradation when the LLM is off.

Contents

- 1 Executive Summary
- 2 System Architecture and Request Flow
- 3 Component-by-Component Code Flow
- 4 Design Decisions and Rationale (interview-ready)
- 5 Production-Readiness: Findings and Fixes
- 6 Security Posture (OWASP-aligned)
- 7 Deploying on AWS
- 8 Interview Preparation: Likely Questions
- 9 How to Run
- 10 Appendix: Configuration Reference

2. System Architecture and Request Flow

The system is a small, layered Flask application. A thin web/API layer validates input and delegates all real work to a **TalentHuntAgent**, which orchestrates a GitHub client, a text/skill utility layer, a deterministic ranker, and a provider-agnostic LLM client. The frontend is server-rendered Jinja2.

The agent pipeline



Every stage degrades gracefully. If no LLM provider is configured, planning falls back to keyword extraction and ranking falls back to the heuristic, so the product still returns useful results with zero external AI dependencies.

Layers and responsibilities

Layer	Responsibility
Web / API (app.py)	Routing, input clamping, rate limiting, security headers, CORS, error mapping, health checks.
Agent (agent.py)	Orchestrates plan -> search -> enrich -> rank; blends heuristic and LLM signals.
GitHub client (github_scraper.py)	HTTP with retries, rate-limit awareness, typed and sanitized errors.
LLM client (llm.py)	Provider abstraction over OpenAI and AWS Bedrock; safe JSON parsing.
Ranker (ranker.py)	Deterministic, explainable keyword-overlap scoring with log-scaled reach.
Utilities (utils.py)	Token-based keyword extraction and alias-driven skill detection.
Config (config.py)	Single source of truth for every tunable; enforces hard ceilings.

3. Component-by-Component Code Flow

config.py - configuration and safety ceilings

All environment access is centralized here. Numeric parsing is defensive, and request sizes are clamped through `clamp_search_size` and `clamp_max_results` so a malicious client cannot ask the server to do unbounded work.

utils.py - keyword and skill extraction

Text is tokenized with a regex that preserves technology punctuation (c++, c#, node.js) while stripping trailing separators. Skills are matched against an alias map by whole token, eliminating the previous false positives where 'c' matched inside 'react' and 'ai' inside 'email'.

scrapers/github_scraper.py - resilient GitHub client

- Bounded retries with exponential backoff for network errors and HTTP 5xx.
- Rate-limit awareness via the Retry-After and X-RateLimit-Reset headers.
- Typed errors (auth, rate-limit, not-found) carrying messages safe to show users.
- Search capped at GitHub's 1000-result ceiling to avoid wasted pagination.

ranker.py - explainable heuristic

Scores combine skill, repository, and bio keyword matches with a log-scaled follower term, so a few very-popular accounts cannot dominate purely on reach. It returns the matched keywords, which makes every score explainable.

llm.py - provider-agnostic LLM client

A tiny client exposes `complete_json` and `complete_text` over either OpenAI or AWS Bedrock. It applies request timeouts, parses JSON tolerantly (handling code fences), and reports availability so callers can fall back instead of failing.

agent.py - the orchestrator

The agent ties everything together. It plans the search (LLM or heuristic), enriches profiles in parallel, pre-ranks with the heuristic, then deep-evaluates the top-K candidates with the LLM and blends the two signals (0.75 LLM fit + 0.25 normalized heuristic). Per-candidate failures fall back to the heuristic so one bad call never breaks a run.

app.py - hardened web and API surface

- ProxyFix so client IP and scheme are read from the AWS load balancer.
- Per-IP rate limiting on the expensive search routes.
- Strict security headers and scoped CORS for the JSON API.
- Error mapping to safe messages and correct HTTP status codes.
- Health endpoints for load-balancer and container checks.

4. Design Decisions and Rationale (interview-ready)

Why a blended heuristic + LLM ranker instead of LLM-only?

The heuristic is free, deterministic, and instant, which makes it ideal for pre-filtering hundreds of profiles down to a shortlist. The LLM is then spent only on the top-K, controlling cost and latency while still delivering nuanced, explainable judgments. Blending keeps a stable baseline signal so a single odd LLM score cannot rank an obviously weak candidate first.

Why a provider abstraction over OpenAI and Bedrock?

The deployment target is AWS, where Bedrock keeps data and billing inside the account, but OpenAI is the most universally available for local development. A one-interface client lets the same code run in both environments via a single environment variable, and makes the LLM a swappable dependency rather than a hard-coded vendor.

Why token-based skill matching rather than substring search?

Substring matching produced false positives ('c' inside 'react'). Tokenizing on word boundaries while preserving technology punctuation, then matching against an alias map, is both precise and tolerant of real-world variants like 'nodejs' or 'golang'.

Why ThreadPoolExecutor for enrichment and evaluation?

The workload is I/O bound (HTTP to GitHub, HTTPS to the LLM). Threads overlap that latency cheaply without the complexity of async, and bounded worker counts keep us well within upstream rate limits.

Why clamp inputs in config rather than trust the form?

The API is public. Centralized clamping guarantees a hard server-side ceiling on work regardless of what a client submits, which is the difference between a feature and a denial-of-service vector.

Why graceful degradation everywhere?

External dependencies fail. Treating the LLM and even individual GitHub calls as optional means a token outage or a single 404 degrades quality slightly instead of returning an error page.

5. Production-Readiness: Findings and Fixes

The table maps each review finding (in original severity order) to the concrete resolution and where it lives in the codebase.

Severity	Finding	Resolution	Location
CRITICAL	Flask debug mode always enabled in production startup.	Debug is now driven by APP_ENV and defaults to off; Gunicorn is the production server.	app.py, config.py
HIGH	Unbounded, expensive work on a public endpoint (DoS / quota abuse).	Hard server-side ceilings plus per-IP rate limiting on search routes.	config.py, app.py
HIGH	Substring skill matching caused ranking false positives.	Token + alias matching with technology-aware tokenization.	utils.py
HIGH	Raw exception and upstream text leaked to clients.	Typed errors, server-side logging, and sanitized user-facing messages.	github_scraper.py, app.py
MEDIUM	Numeric input parsing could throw and return 500.	Defensive parse-and-clamp helpers; invalid input is coerced safely.	config.py
MEDIUM	External target=_blank links lacked rel protection.	Added rel='noopener noreferrer' to all external links.	results.html
MEDIUM	Unpinned dependencies; an unused library was shipped.	Versions pinned; beautifulsoup4 removed; production and LLM deps added.	requirements.txt
MEDIUM	Misconfigured .gitignore (secret not ignored).	Corrected to ignore .env and caches while keeping .env.example.	.gitignore
MEDIUM	Incomplete production serving model.	Gunicorn config, WSGI entrypoint, Docker image, health checks.	wsgi.py, gunicorn.conf.py, Dockerfile
LOW	No automated tests for core behavior.	Unit tests for skill detection, ranking, and input clamping.	tests/

6. Security Posture (OWASP-aligned)

Risk	Mitigation
A01 Broken Access Control	Scoped CORS; the API exposes only what it needs; no write surface.
A03 Injection	No SQL; user input flows into parameterized GitHub queries and escaped Jinja output.
A05 Security Misconfiguration	Debug off by default; strict security headers; least-privilege container user.
A06 Vulnerable Components	Pinned dependencies enable reproducible builds and scanning.
A07 Auth Failures	Secrets via environment / AWS Secrets Manager; tokens never committed or logged.
A09 Logging Failures	Structured logs to stdout; upstream error bodies logged server-side only.
Abuse / DoS	Per-IP rate limiting plus hard request ceilings on expensive routes.

Cross-site scripting is mitigated by Jinja2 autoescaping and a Content-Security-Policy that disables scripting (script-src none), since the UI ships no JavaScript.

7. Deploying on AWS

The recommended path is a container on ECS Fargate behind an Application Load Balancer. The image is built from the repository root Dockerfile and runs Gunicorn.

Build and push the image

```
docker build -t talenthunt .
aws ecr create-repository --repository-name talenthunt
aws ecr get-login-password | docker login --username AWS --password-stdin
ACCOUNT.dkr.ecr.REGION.amazonaws.com
docker tag talenthunt ACCOUNT.dkr.ecr.REGION.amazonaws.com/talenthunt:latest
docker push ACCOUNT.dkr.ecr.REGION.amazonaws.com/talenthunt:latest
```

Run on ECS Fargate

- 1 Create an ECS cluster and a Fargate task definition referencing the pushed image.
- 2 Map container port 8000 and set the ALB target-group health check to /health.
- 3 Inject configuration as task environment variables (see the appendix).
- 4 Store GITHUB_TOKEN, SECRET_KEY, and OPENAI_API_KEY in AWS Secrets Manager and reference them as secrets in the task definition.
- 5 Attach an autoscaling policy on CPU or request count for elasticity.

Using AWS Bedrock for the LLM

- Set LLM_PROVIDER=bedrock and BEDROCK_MODEL_ID to an enabled model.
- Grant the task role bedrock:InvokeModel on that model ARN.
- No API key is needed; credentials flow through the task role.

Operational notes

- Use a Redis-backed RATELIMIT_STORAGE_URI so limits are shared across tasks.
- Gunicorn logs to stdout, which CloudWatch collects automatically.
- Set WEB_CONCURRENCY and GUNICORN_THREADS to match the task's vCPU.

8. Interview Preparation: Likely Questions

How would you scale this to thousands of searches per minute?

Make the agent stateless (it already is), run many Fargate tasks behind the ALB, move rate-limit state to Redis, cache GitHub profiles with a short TTL, and push long-running hunts onto a queue (SQS) with results delivered asynchronously.

How do you control LLM cost and latency?

Pre-filter with the free heuristic and only evaluate the top-K; cap output tokens; use a small fast model (gpt-4o-mini or Claude Haiku); evaluate candidates concurrently with a bounded thread pool; cache evaluations by (profile, role).

What happens when GitHub rate-limits you?

The client honors Retry-After for short waits and otherwise raises a typed rate-limit error mapped to HTTP 429. A configured GITHUB_TOKEN raises the ceiling from 60 to 5000 requests per hour.

How do you keep ranking fair and explainable?

The heuristic exposes matched keywords, and the LLM returns a short rationale plus matched and missing skills. The evaluation prompt explicitly avoids speculation about protected attributes.

How would you test the LLM behavior?

Inject a fake LLM client implementing the same interface to assert blending and fallback logic deterministically, and contract-test the JSON parsing against messy model outputs.

9. How to Run

Local development

```
cd backend
python -m venv .venv and activate it
pip install -r requirements.txt
copy .env.example to .env and fill in values
python app.py --debug # development server
gunicorn -c gunicorn.conf.py wsgi:app # production server
```

Command-line interface

```
python main.py -q "python backend django" -l "Berlin" -n 10 -s 100
```

JSON API

```
POST /api/search
{ "query": "rust systems engineer", "location": "remote",
  "max_results": 10, "search_size": 100 }
```

Generate this report

```
python generate_report.py
```

10. Appendix: Configuration Reference

Variable	Default	Purpose
APP_ENV	production	production disables debug and verbose errors
GITHUB_TOKEN	(none)	raises GitHub rate limit to 5000/hour
MAX_SEARCH_SIZE	100	hard ceiling on users scanned per request
MAX_RESULTS	50	hard ceiling on returned candidates
SCRAPER_MAX_WORKERS	8	thread pool size for profile enrichment
RATELIMIT_SEARCH	10 per minute	per-IP limit on search routes
RATELIMIT_STORAGE_URI	memory://	use redis:// for multi-instance limits
LLM_PROVIDER	openai	one of none, openai, bedrock
LLM_EVAL_TOP_K	12	candidates deeply evaluated by the LLM
OPENAI_MODEL	gpt-4o-mini	OpenAI model id
BEDROCK_MODEL_ID	claude-3-haiku	Bedrock model id when provider=bedrock

End of report. Generated with ReportLab.